Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: ”Informații invizibile: forensics și steganografie”

A efectuat : st. gr.SI – 263; Topală Dorofei
A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

1. Lucrarea A:

- Studierea diferenței dintre text (str) și octeți (bytes) în Python;

- Implementarea funcției de clasificare a codificărilor (URL, Hex, Base64);

- Decodificarea straturilor succesive până la obținerea textului citibil;

- Spargerea prin forță brută a unui text criptat cu XOR pe un octet;

- Spargerea unui hash de parolă prin atac de dicționar pe baza lungimii acestuia;

- Salvarea rezultatelor analizei într-o fișă JSON și validarea decodorului prin teste unitare.

2. Lucrarea B:

- Identificarea tipului real al fișierelor pe baza semnăturilor de început (Magic Bytes);

- Extragerea secvențelor de caractere imprimabile din fișiere binare;

- Parsarea metadatelor EXIF și conversia coordonatelor GPS în grade zecimale;

- Extragerea fișierelor ascunse (File Carving) prin căutarea semnăturii de arhivă ZIP;

- Implementarea algoritmului de steganografie LSB (Least Significant Bit) pentru ascunderea și dezvăluirea unui mesaj text în pixelii unei imagini fără pierderi.

LUCRAREA A: Decoderul universal: arsenal/forensics/decode.py

A1. Text și octeți

În Python, este important să deosebim textul (str) de octeți (bytes — numere de la 0 la 255).

- Ca să transformi textul în octeți, folosești metoda.encode().
- Ca să vezi fiecare octet în baza 16, folosești formatul hexazecimal.

Implementare cod Python:

```
s = "Salut"
s_bytes = s.encode()
print("bytes:", s_bytes, "  b[0]:", s_bytes[0])
print("hex:", s_bytes.hex())
print("din hex:", bytes.fromhex(s_bytes.hex()))
```

Rezultat în terminal:

```
bytes: b'Salut'   b[0]: 83
hex: 53616c7574
din hex: b'Salut'
```

A2. Recunoașterea stratului după formă

Pentru a automatiza analiza datelor obfuscate, a fost implementată funcția ghici_strat(), care scanează trăsăturile structurale ale șirului de intrare prin expresii regulate:

- **URL-encoding:** identificat prin secvențe de tip %XX (unde XX sunt cifre hexazecimale).
- **Hexazecimal:** conține exclusiv caractere din setul [0-9a-fA-F] și are lungimea strict pară.
- **Base64:** respectă alfabetul [A-Za-z0-9+/=] și are lungimea multiplu de 4.

Implementare cod Python:

```
import re

def ghici_strat(s):
    if "%" in s and re.search(r"%[0-9A-Fa-f]{2}", s):
        return "url"
    if re.fullmatch(r"[0-9a-fA-F]+", s) and len(s) % 2 == 0:
        return "hex"
    if re.fullmatch(r"[A-Za-z0-9+/=]+", s) and len(s) % 4 == 0:
        return "base64"
    return "necunoscut"
```

Rezultat teste în terminal:

```
ghici_strat('%2Fetc'): url
ghici_strat('48656c6c6f'): hex
ghici_strat('U2FsdXQ='): base64
```

A.3 Desfacerea stratului recunoscut

Funcția desfa() primește șirul codat, interoghează ghici_strat() și execută decodificarea corespunzătoare prin modulele base64 sau urllib.parse:

```
import base64
from urllib.parse import unquote

def desfa(s):
    strat = ghici_strat(s)
    if strat == "base64":
        return strat, base64.b64decode(s)
    elif strat == "hex":
        return strat, bytes.fromhex(s)
    elif strat == "url":
        return strat, unquote(s).encode()
    return strat, s.encode()
```

Rezultat în terminal:

```
desfa('U2FsdXQ='): ('base64', b'Salut')
desfa('48656c6c6f'): ('hex', b'Hello')
```

A4. Straturi multiple, într-o buclă

Fișierul probe/mesaj.txt a fost analizat iterativ. La fiecare pas a fost extras un strat de codificare până când toți octeții au aparținut intervalului de caractere ASCII imprimabile (32...126).

```
def citibil(b):
    return all(32 <= c < 127 for c in b)
def decodare_completa(val):
    """Desfasoara toate straturile pana la text citibil."""
    straturi = []
    for _ in range(8):
        strat, b = desfa(val)
        if strat == "necunoscut":
            break
        straturi.append(strat)
        try:
            val_nou = b.decode()
        except UnicodeDecodeError:
            break
        if citibil(b) and ghici_strat(val_nou) == "necunoscut":
            return straturi, b
        val = val_nou
    return straturi, b
```

Rezultat în terminal:

```
Mesaj codat: NTM2NTYzNzU3MjY5NzQ2MTc0NjU2MTIwNjk2ZTYzNjU3MDY1MjA2Mzc1MjA2...
Strauri gasite: ['url', 'base64', 'hex']
Text final: Securitatea incepe cu curiozitatea
```

Analiza arată că mesajul a fost codat în trei straturi succesive: **URL → Base64 → Hex**. Textul recuperat este: *"Securitatea incepe cu curiozitatea"*.

A5. Spargere XOR

Operația logică XOR este des întâlnită în schemele criptografice elementare. Atunci când cheia de criptare constă dintr-un singur octet, spațiul cheilor este restrâns la 256 variante, făcând posibil atacul prin forță brută. Criptograma din probe/xor.bin a fost decriptată prin parcurgerea exhaustivă a fiecărui octet și verificarea caracterelor ASCII lizibile ($32 \le c < 127$).

Implementare cod Python:

```
def sparge_xor(cale):
    date = open(cale, "rb").read()
    candidati = []
    for k in range(256):
        clar = bytes(b ^ k for b in date)
        if citibil(clar):
            text = clar.decode()
            nr_litere = sum(c.isalpha() for c in text)
            nr_spatii = text.count(" ")
            candidati.append((nr_litere + nr_spatii * 5, k, text))
    if candidati:
        candidati.sort(reverse=True)
        _, k, text = candidati[0]
        return k, text Dezvălui mesajul înapoi
    return None, None
```

Rezultat în terminal:

```
Cheie gasita: 42
Text: Forensics dezvaluie ce ochiul nu vede
```

Cheia identificată cu succes este **42** (reprezentare hexazecimală 0x2A), iar mesajul descifrat este: *"Forensics dezvaluie ce ochiul nu vede"*.

A6. Spargerea hash-ului prin atac cu dicționar

Pentru inversarea unei amprente unidirecționale, în practică se apelează la atacul de dicționar (*dictionary attack*). Din fișierul `probe/hashuri.txt` a fost extras hash-ul țintă cu o lungime de 32 caractere hexazecimale, caracteristic algoritmului MD5 (128 biți). Fiecărui cuvânt din fișierul `probe/wordlist.txt` i s-a calculat hash-ul MD5 până la identificarea potrivirii.

Implementare cod Python:

```
def sparge_hash(cale_hash, cale_wordlist):
    tinta = open(cale_hash).readline().strip()
    lung = len(tinta)

    if lung == 32:
        alg = "md5"
    elif lung == 40:
        alg = "sha1"
    elif lung == 64:
        alg = "sha256"
    else:
        return None, None, None

    for cuv in open(cale_wordlist, encoding="latin1"):
        cuv = cuv.strip()
```

Rezultat în terminal:

```
Algoritm: md5
Hash: 8287458823facb8ff918dbfabcd22ccb
Parola gasita: parola
```

2.3.

A7. Serializarea rezultatelor în format JSON și testarea unitară

Conform cerințelor de modularitate, rezultatele investigației au fost salvate în fișierul tehnic arsenal/forensics/fisa.json, iar acuratețea algoritmilor a fost validată cu suita de teste automate teste/test_decode.py.

Conținut fișier JSON (arsenal/forensics/fisa.json):

```
    print("=== A7. Salvare fisa JSON ===")
    fisa = {
        "mesaj_codat": {
            "straturi": straturi,
            "rezultat": rezultat.decode() if isinstance(rezultat, bytes)
else str(rezultat)
        },
        "xor": {
            "cheie": cheie,
            "text": text_xor
        },
        "hash": {
            "algoritm": alg,
            "hash": hash_val,
            "parola": parola
        }
    }
    with open("arsenal/forensics/fisa.json", "w", encoding="utf-8") as f:
        json.dump(fisa, f, indent=2, ensure_ascii=False)
    print("Fisa salvata in arsenal/forensics/fisa.json")
    print(json.dumps(fisa, indent=2, ensure_ascii=False))
```

Rezultat execuție teste unitare:

```
test_base64 OK
test_hex OK
test_url OK
test_citibil OK
test_ghici_strat OK
Toate testele au trecut!
```

LUCRAREA B (extindere). Forensics pe fișiere și un mesaj ascuns

B1. Identificarea tipului real al fișierelor prin Magic Bytes

În investigațiile digitale, extensia unui fișier este nesigură, putând fi alterată cu ușurință. Determinarea certă a formatului se realizează prin inspectarea semnăturilor binare de început (*magic bytes*). Funcția tip_real() compară primii octeți cu semnăturile recunoscute internațional:

Implementare cod Python:

```
def tip_real(cale):
    cap = open(cale, "rb").read(8)
    if cap[:4] == b"%PDF":
        return "pdf"
    if cap[:4] == b"\x89PNG":
        return "png"
    if cap[:2] == b"\xff\xd8":
        return "jpeg"
    if cap[:4] == b"PK\x03\x04":
        return "zip/docx"
    return cap.hex()
```

Rezultat în terminal:

```
foto.jpg: jpeg
ascuns.png: png
```

B2. Extragerea șirurilor imprimabile și detecția anomaliilor

Prin scanarea conținutului brut al fișierului probe/ascuns.png cu o expresie regulată aplicată pe fluxul de octeți, s-au extras secvențele de caractere imprimabile:

Implementare cod Python:

```
def extrage_strings(cale, minim=4):
    date = open(cale, "rb").read()
    rezultate = []
    for m in re.findall(rb"[ -~]{" + str(minim).encode() + rb",}", date):
        rezultate.append(m.decode())
    return rezultate
```

Rezultat în terminal:

```
  IHDR
  /IDATx
  IEND
  secret.txtFelicitari! Ai gasit fisierul ascuns in imagine.
  secret.txtPK
```

Analiza relevă prezența numelui secret.txt și a antetului PK după chunk-ul de închidere al formatului PNG (IEND), semnalând existența unei arhive ZIP camuflate prin concatenare.

B3. Metadate EXIF și GPS din poză

Imaginile capturate cu dispozitive foto moderne conțin metadate în format EXIF. Utilizând biblioteca PIL.ExifTags, au fost extrase detaliile tehnice ale camerei și coordonatele GPS din fișierul probe/foto.jpg.

Implementare cod Python:

```python
def citeste_exif(cale):
    img = Image.open(cale)
    exif = img._getexif() or {}
    info = {}
    for id_tag, val in exif.items():
        nume = TAGS.get(id_tag, id_tag)
        info[nume] = val
    return info
```

```python
def gps_grade(coord, ref):
    """Converteste coordonate GPS din (grade, minute, secunde) in grade
zecimale."""
    d = float(coord[0])
    m = float(coord[1])
    s = float(coord[2])
    grade = d + m / 60.0 + s / 3600.0
    if ref in (b"S", b"W", "S", "W"):
        grade = -grade
    return round(grade, 6)
```

```python
def citeste_gps(cale):
    img = Image.open(cale)
    exif = img._getexif() or {}
    gps_data = exif.get(34853, {})
    if not gps_data:
        return None, None

    lat = gps_grade(gps_data[2], gps_data[1])
    lon = gps_grade(gps_data[4], gps_data[3])
    return lat, lon
```

Rezultat în terminal:

```
Model = Forensics-6
Software = Python PIL
Make = LabCamera

GPS: 47.0105 N, 28.8638 E
Link harta: https://www.google.com/maps?q=47.0105,28.8638
```

Coordonatele geografice decodificate indică locația exactă: **Chișinău, Republica Moldova** ($47.0105^\circ ext{N}, 28.8638^\circ ext{E}$).

B4. Decuparea fișierelor ascunse (File Carving)

Tehnica de *file carving* permite extragerea fișierelor independente dintr-un flux de date continuu. Identificând secvența de octeți specifică arhivei ZIP (PK) în interiorul imaginii probe/ascuns.png, datele au fost extrase și scrise în fișierul gasit.zip.

Implementare cod Python:

```python
def extrage_ascuns(cale, output="gasit.zip"):
    date = open(cale, "rb").read()
    i = date.find(b"PK\x03\x04")
    if i != -1:
        open(output, "wb").write(date[i:])
        return i, output
```

Rezultat în terminal:

```
Arhiva gasita la octetul 360
Continut arhiva: ['secret.txt']
[secret.txt]: Felicitari! Ai gasit fisierul ascuns in imagine.
```

Arhiva începe la offset-ul de octet **360**. La dezarhivare, fișierul secret.txt dezvăluie mesajul:*"Felicitari! Ai gasit fisierul ascuns in imagine."*

B5. Camuflarea mesajului în imagine (LSB Encoding)

Steganografia LSB (*Least Significant Bit*) substituie cel mai puțin semnificativ bit al fiecărui canal de culoare (Roșu, Verde, Albastru) al pixelilor unei imagini cu biții mesajului secret. Modificarea valorii unui canal este de maximum ± 1 din 255, variație imperceptibilă pentru sistemul vizual uman. A fost încorporat mesajul FLAG{ascuns_in_pixeli} în imaginea probe/foto.jpg, generându-se fișierul stego.png.

Implementare cod Python:

```
from PIL import Image

img = Image.open("probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}"
biti = "".join(f"{x:08b}" for x in mesaj)

px_nou = []
idx_bit = 0
for (r, g, b) in px:
    canale = [r, g, b]
    for i in range(3):
        if idx_bit < len(biti):
            canale[i] = (canale[i] & ~1) | int(biti[idx_bit])
            idx_bit += 1
    px_nou.append(tuple(canale))

img.putdata(px_nou)
img.save("stego.png", "PNG")
```

Rezultat:

```
Mesaj ascuns: FLAG{ascuns_in_pixeli}
Imagine salvata: stego.png
```

B6.Dezvălui mesajul înapoi

Procesul de recuperare constă în parcurgerea pixelilor din stego.png, izolarea bitului LSB pentru fiecare canal de culoare prin mascare logică (canal & 1), gruparea biților în octeți și conversia acestora în caractere până la octetul nul sau finalul mesajului.

Implementare cod Python:

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    biti += str(r & 1)
    biti += str(g & 1)
    biti += str(b & 1)

octeti = []
for i in range(0, len(biti), 8):
    octet = int(biti[i:i+8], 2)
    if octet == 0:
        break
    octeti.append(octet)
mesaj = bytes(octeti)
```

Rezultat în terminal:

```
Mesaj gasit: FLAG{ascuns_in_pixeli}
Verificare: OK
```

CONCLUZII

În cadrul acestei lucrări de laborator am aprofundat conceptele fundamentale de analiză forensic digitală, criptanaliză aplicată și steganografie pe imagini, dobândind abilități practice solide de manipulare a datelor la nivel binar în limbajul de programare Python. Am proiectat și implementat cu succes un decodificator universal autonom capabil să inverseze scheme multiple de codare (URL, Base64, Hexazecimal) și să execute atacuri de tip *brute-force* pe chei XOR de un octet și *dictionary attack* pe amprente criptografice MD5. Totodată, am asimilat tehnici vitale de investigație digitală, cum ar fi validarea integrității fișierelor prin semnături de tip *magic bytes*, localizarea pe hartă pe baza metadatelor EXIF/GPS, extragerea conținutului ascuns prin tehnici de *file carving* și aplicarea metodei LSB pentru ascunderea nedetectabilă a canalelor de comunicație secrete în imagini grafice. Rezultatele practice obținute evidențiază rolul critic al analizei fluxurilor de octeți în securitatea sistemelor informatice moderne.